Proyecto: DondeE

ICI 324 - Bases de Datos y Programación Web
Grupo 2

Martín Araya Riquelme Jorge Bahamondes Amador
Bruno Díaz Fernández

20 de octubre de 2026

1. Introducción

1.1. Descripción del problema a abordar

La difusión de eventos y la promoción de la oferta turística y de esparcimiento nocturno se encuentra actualmente polarizada. Las grandes productoras y cadenas de entretenimiento acaparan la visibilidad en medios digitales y redes sociales tradicionales gracias a sus elevados presupuestos publicitarios. Esto deja a los locales nocturnos de menor y mediano tamaño (pubs, bares y discotecas independientes) sin un canal de difusión ágil, geolocalizado y eficiente para dar a conocer sus actividades en tiempo real a clientes potenciales que se encuentran en su cercanía geográfica.

Por otro lado, la proliferación de eventos informales y no regulados genera problemas de seguridad para los asistentes e incide en la falta de fiscalización. Para mitigar estos riesgos y asegurar la viabilidad legal y social de la plataforma, el sistema se enfoca exclusivamente en dinamizar y visibilizar la economía nocturna formal y regulada.

1.2. Afectados y Beneficiarios

- **Locales comerciales nocturnos (Host Comercial):** Bares, pubs y discotecas formales que requieren validar legalmente sus permisos municipales y de patente de alcoholes. Se benefician al disponer de un canal directo para captar público espontáneo cercano a su ubicación, reduciendo significativamente los costos de publicidad masiva tradicional.

- **Usuarios Asistentes (Clientes):** Personas que buscan opciones de recreación seguras, verificadas y consolidadas en tiempo real. Se benefician al poder tomar decisiones informadas a través de un mapa interactivo con filtros geoespaciales basados en su posición GPS actual, garantizando que los locales sugeridos cumplan con la normativa legal vigente.

- **Administradores de la plataforma:** Entidades encargadas del control de calidad, moderación y validación de la documentación legal (patentes y licencias) de los locales registrados, asegurando un ecosistema confiable y seguro para la comunidad.

1.3. Objetivo de Negocio y Propuesta de Solución

El objetivo de negocio es potenciar la economía nocturna formal mediante una plataforma web integral que optimice el descubrimiento de oferta recreativa local en tiempo real y rentabilice la conversión de asistentes mediante la gestión segura de reservas y aforos.

Para dar respuesta a esta necesidad, la propuesta de solución (actualmente en Etapa 1 de diseño y prototipado) consiste en una plataforma web *full-stack* respaldada por un motor de base de datos relacional con soporte de extensión espacial (PostGIS sobre PostgreSQL). La aplicación combinará el descubrimiento geoespacial interactivo en tiempo real con la gestión transaccional de reservas y accesos.

A diferencia de las plataformas de redes sociales genéricas, donde la información de eventos suele ser estática y depende de algoritmos de alcance pagado, esta solución utilizará la ubicación espacial precisa del usuario para presentar oferta disponible en un radio geográfico inmediato. Asimismo, el diseño de la plataforma impone como requisito técnico obligatorio que todo usuario organizador autentique y asocie un recinto autorizado con

su documentación municipal al día antes de publicar cualquier evento, garantizando así la seguridad del usuario y la legitimidad de la oferta publicitaria.

2. Requerimientos Propuestos

Nota: Los siguientes requerimientos detallan el diseño arquitectónico y funcional esperado para la versión final del sistema. La implementación actual (Etapa 1) incluye la base de datos completa y prototipos con datos simulados (mocks) para validar la viabilidad del flujo de usuario y endpoints backend.

2.1. Clases de Usuarios del Sistema

1. **Usuario Asistente (Cliente):** Persona consumidora final que utiliza la plataforma para explorar el mapa interactivo, buscar eventos formales cercanos mediante su ubicación GPS, revisar la información y comentarios del recinto, y adquirir o reservar accesos en tiempo real.

2. **Host Comercial (Organizador):** Creador de eventos correspondiente a dueños de locales nocturnos (pubs, bares, discotecas) o productoras autorizadas. Poseen privilegios para gestionar el perfil de su recinto, registrar documentación legal (patentes y licencias) y crear o modificar eventos dentro de su aforo máximo permitido.

3. **Administrador:** Encargado de la fiscalización y moderación global de la plataforma. Su función principal es validar la documentación legal (patentes y licencias) de los recintos comerciales antes de otorgarles permisos de publicación, asegurando la legitimidad del ecosistema.

2.2. Requerimientos Funcionales

- **RF01 - Gestión de Identidad y Roles:** El sistema debe permitir a los usuarios registrarse, iniciar sesión y recuperar su contraseña según su rol asignado (*Usuario Asistente*, *Host Comercial* o *Administrador*).

- **RF02 - Publicación de Eventos Formales:** El sistema debe permitir a un Host Comercial verificado crear y publicar un evento asociado a un recinto autorizado, definiendo nombre, fecha/hora, precio de entrada, categoría musical, aforo máximo y ubicación geográfica.

- **RF03 - Filtrado Geo-Espacial:** El sistema debe proveer un mapa interactivo donde el usuario pueda consultar eventos cercanos aplicando obligatoriamente su ubicación GPS y un radio de distancia en kilómetros, con posibilidad de aplicar filtros opcionales (categoría del local, rango de precios, estilo musical y fecha).

- **RF04 - Transacción de Reserva/Compra:** El sistema debe permitir a un usuario reservar un cupo o comprar una entrada emitiendo un ticket digital y descontando la disponibilidad del aforo en tiempo real.

- **RF05 - Gestión de Reseñas y Valoraciones:** El sistema debe permitir a los usuarios que asistieron a un evento publicar calificaciones y comentarios textuales sobre el recinto, así como consultar la retroalimentación de otros usuarios.

2.3. Requerimientos No Funcionales

- **RNF01 - Rendimiento Geo-espacial:** Las búsquedas por radio de distancia en el mapa deben ejecutarse en menos de 500 ms utilizando indexación espacial (ej. PostGIS / GiST) a nivel de base de datos. *(Meta a medir en siguientes etapas).*
- **RNF02 - Seguridad y Autenticación:** Implementación de autenticación segura stateless mediante tokens (JWT) y contraseñas encriptadas con algoritmos de hashing robustos (ej. bcrypt), complementado con control de acceso basado en roles (RBAC).
- **RNF03 - Integridad Transaccional (ACID):** Control estricto a nivel de motor de base de datos al momento de reservar entradas para garantizar operaciones atómicas y evitar sobreventa de aforos (*overbooking*) ante concurrencia simultánea.
- **RNF04 - Usabilidad y Diseño Responsivo:** La interfaz debe seguir un enfoque *Mobile-First*, asegurando tiempos de carga inicial inferiores a 2 segundos en redes móviles y fácil usabilidad táctil. *(Meta a medir en siguientes etapas).*

2.4. Descripción de Casos de Uso de Alto Nivel

- **CU01 - Consulta de Eventos Cercanos (Caso Consulta):** Un usuario asistente busca opciones de esparcimiento nocturno. El sistema le exige ingresar **2 filtros obligatorios** (Ubicación GPS actual y Radio de búsqueda en kilómetros). Opcionalmente, el usuario puede refinar la búsqueda utilizando **4 filtros opcionales** (Categoría del local, Rango de precios, Fecha del evento y Calificación mínima). El sistema despliega marcadores en un mapa interactivo indicando los locales activos. Al seleccionar un marcador, el sistema muestra la información detallada del recinto, incluyendo comentarios en tiempo real de otros usuarios en el lugar.
- **CU02 - Reserva de Cupo y Control de Aforo (Caso Transaccional):** Tras revisar la información de un evento y los comentarios de otros asistentes, el usuario decide asistir y solicita un cupo o compra una entrada. El sistema procesa la solicitud generando un registro de transacción que relaciona 3 entidades principales (*Usuario Asistente*, *Evento* y *Reserva/Ticket*). Automáticamente, la base de datos descuenta el cupo reservado del aforo total permitido para el local, asegurando la integridad de los datos y evitando la sobreventa.
- **CU03 - Mantenedor de Perfil Comercial y Eventos (CRUD):** El usuario con rol de dueño de local (Host Comercial) accede a su panel de administración. A través de este módulo, el usuario es capaz de **crear** nuevos eventos, **modificar** información y especificaciones de eventos futuros (dentro de los margins legales permitidos), **consultar** métricas y feedback de eventos pasados, y **eliminar** eventos cancelados o dar de baja el perfil completo de la empresa.
- **CU04 - Mantenedor de Cuenta de Usuario y Reseñas (CRUD):** El usuario asistente accede a la gestión de su cuenta en la plataforma. Desde aquí, el usuario puede registrarse en el sistema (**crear** perfil), actualizar su información personal o preferencias de búsqueda (**modificar**), revisar su historial de eventos asistidos o comentarios publicados (**consultar**) y borrar reseñas antiguas o darse de baja de la plataforma (**eliminar**).

2.5. Diagramas de Casos de Uso y Secuencia

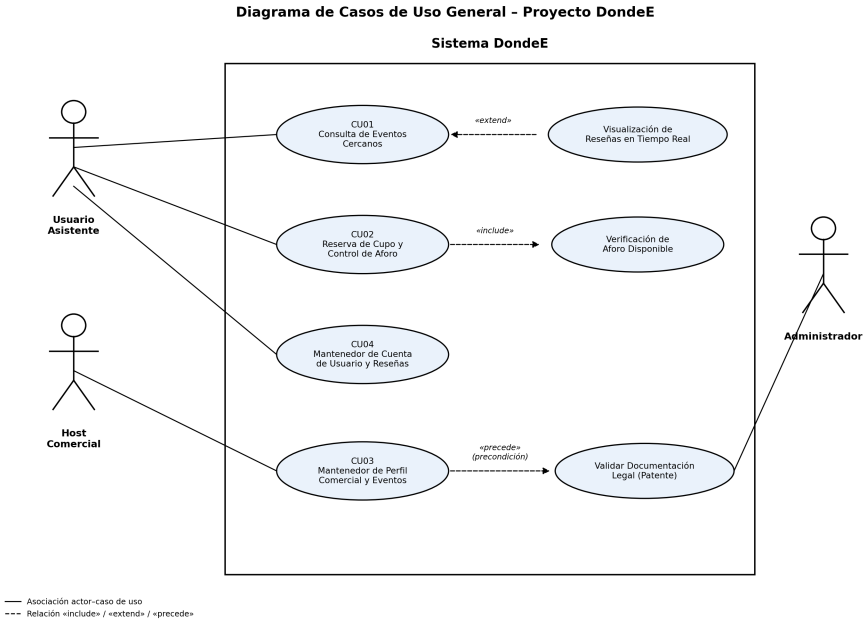


Figura 1: Diagrama de Casos de Uso General - Proyecto DondeE

Justificación de Relaciones entre Casos de Uso:

- **Inclusión («*include*»):** El **CU02 (Reserva de Cupo)** incluye obligatoriamente el caso de uso *Verificación de Aforo Disponible*, ya que no es posible procesar una transacción sin antes validar la disponibilidad del recinto a nivel de base de datos.
- **Extensión («*extend*»):** El **CU01 (Consulta de Eventos)** es extendido de forma opcional por la *Visualización de Reseñas en Tiempo Real*, permitiendo que el usuario explore el mapa sin estar obligado a consultar las opiniones de cada local.
- **Precondición de Dominio:** La ejecución del **CU03 (Publicación de Eventos)** exige como precondición la previa aprobación y validación de la patente municipal del recinto por parte del *Administrador*.

2.6. Prototipo de Casos de Uso y Justificación de Diseño

Justificación de Diseño:
El diseño de la interfaz responde directamente a los Requerimientos Funcionales y No Funcionales especificados. Para la vista de consulta (**CU01**), se adoptó un enfoque *Mobile-First* priorizando un mapa interactivo a pantalla completa que reduce la carga cognitiva. Los controles flotantes de los dos filtros obligatorios (Ubicación GPS y Radio de búsqueda) se ubicaron al alcance táctil directo para acelerar las búsquedas y cumplir con el **RNF04 (Usabilidad)**.

Para el flujo transaccional (**CU02**), se diseñó un flujo modal en 2 pasos que visibiliza de forma transparente los cupos de aforo disponibles en tiempo real antes de requerir la confirmación del usuario. Esta estructura previene errores involuntarios y se alinea con el **RNF03 (Integridad Transaccional)**, garantizando que la reserva se procese con

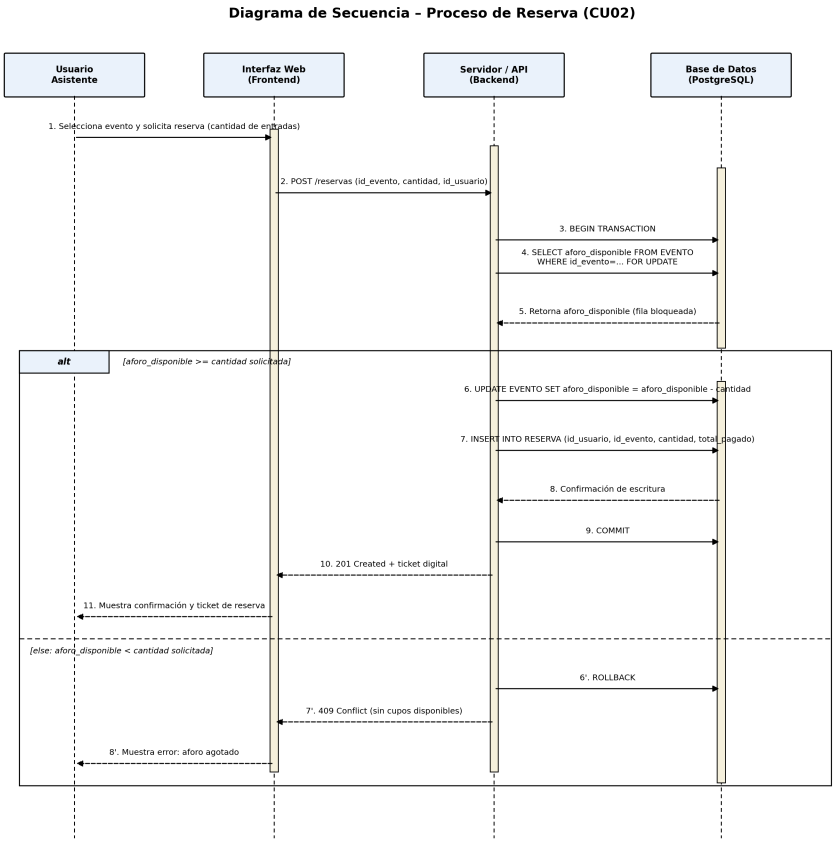


Figura 2: Diagrama de Secuencia Propuesto - Flujo Transaccional Esperado

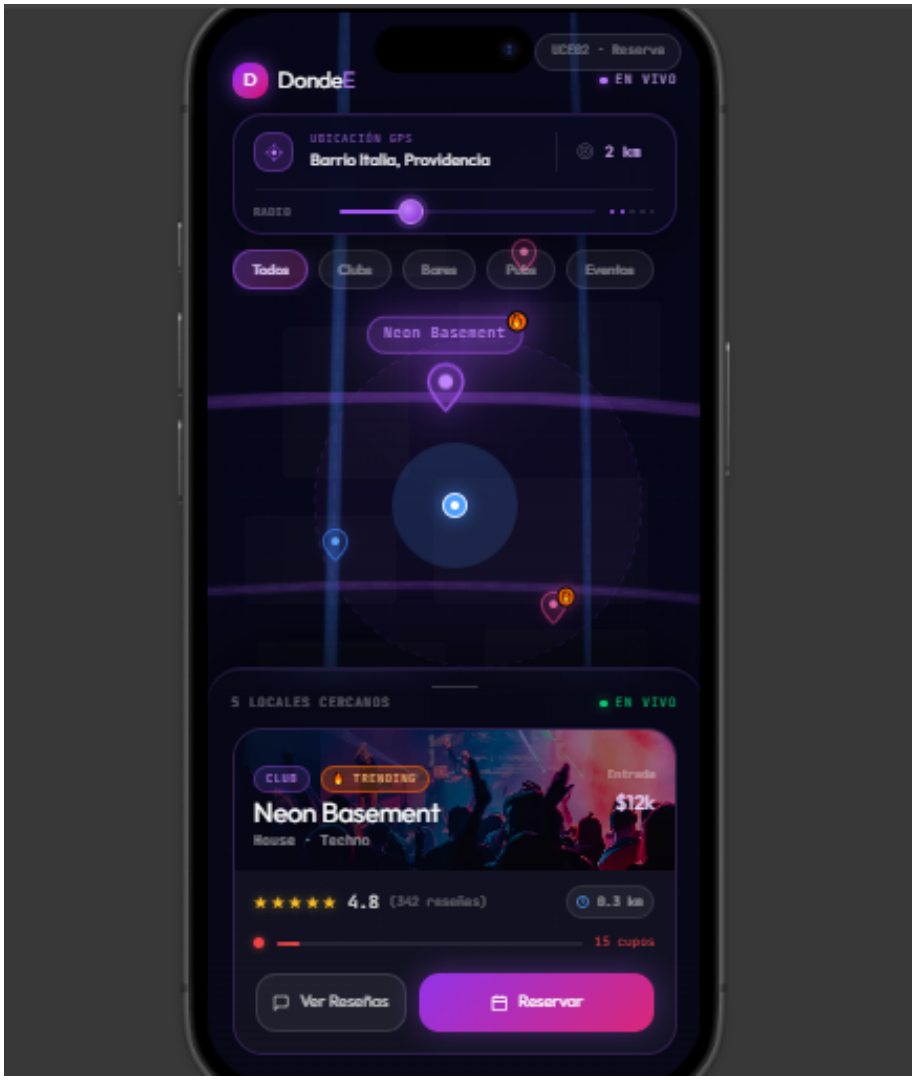


Figura 3: Mockup Conceptual - CU01: Consulta de Eventos

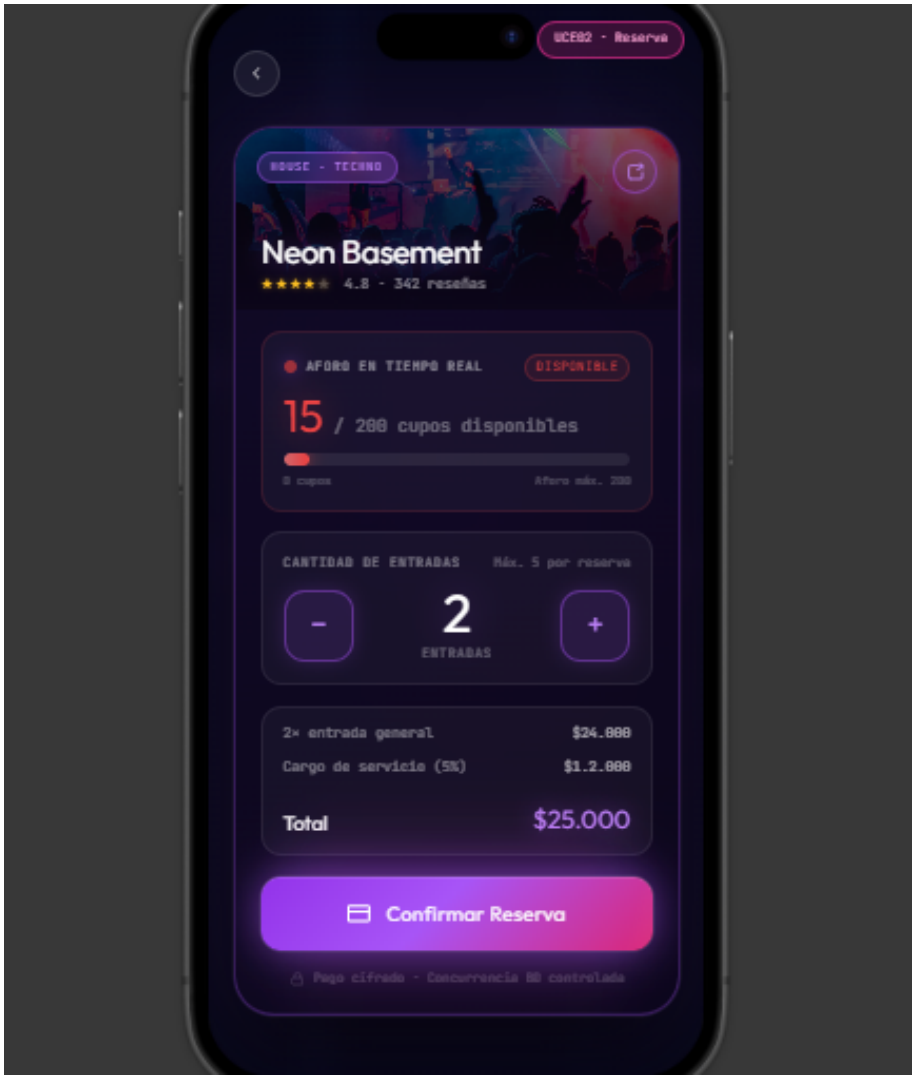


Figura 4: Mockup Conceptual - CU02: Reserva y Pago (Transaccional)

claridad visual antes del descuento atómico en el servidor (flujo esperado para la versión final).

3. Modelo de Datos

3.1. Modelo Entidad-Relación (ER)

El Modelo Entidad-Relación (Figura 5) representa conceptualmente la estructura del sistema, definiendo la interrelación entre los usuarios, recintos, eventos, reservas y valoraciones.

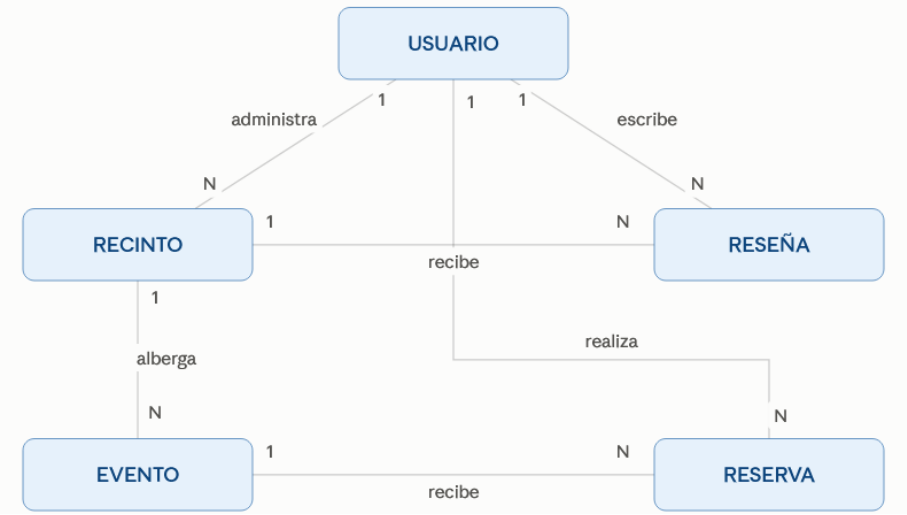


Figura 5: Modelo Entidad-Relación Conceptual

3.2. Modelo Relacional

A partir del diagrama ER, se realiza la transformación al modelo relacional lógico. Las entidades principales se traducen en las siguientes tablas normalizadas (3FN):

- **USUARIO** (id_usuario, nombre, email, password_hash, rol, fecha_registro)
- **RECINTO** (id_recinto, nombre_local, direccion, ubicacion_geom, patente_municipal, estado_validacion, *id_host*)
- **EVENTO** (id_evento, titulo, descripcion, fecha_hora, precio_entrada, categoria_musical, aforo_total, aforo_disponible, *id_recinto*)
- **RESERVA** (id_reserva, fecha_reserva, estado_pago, cantidad_entradas, total_pagado, *id_usuario*, *id_evento*)
- **RESEÑA** (id_reseña, calificacion, comentario, fecha_publicacion, *id_usuario*, *id_recinto*)

3.3. Diccionario de Datos

A continuación, se detalla la estructura física de la base de datos relacional ajustada al esquema DDL:

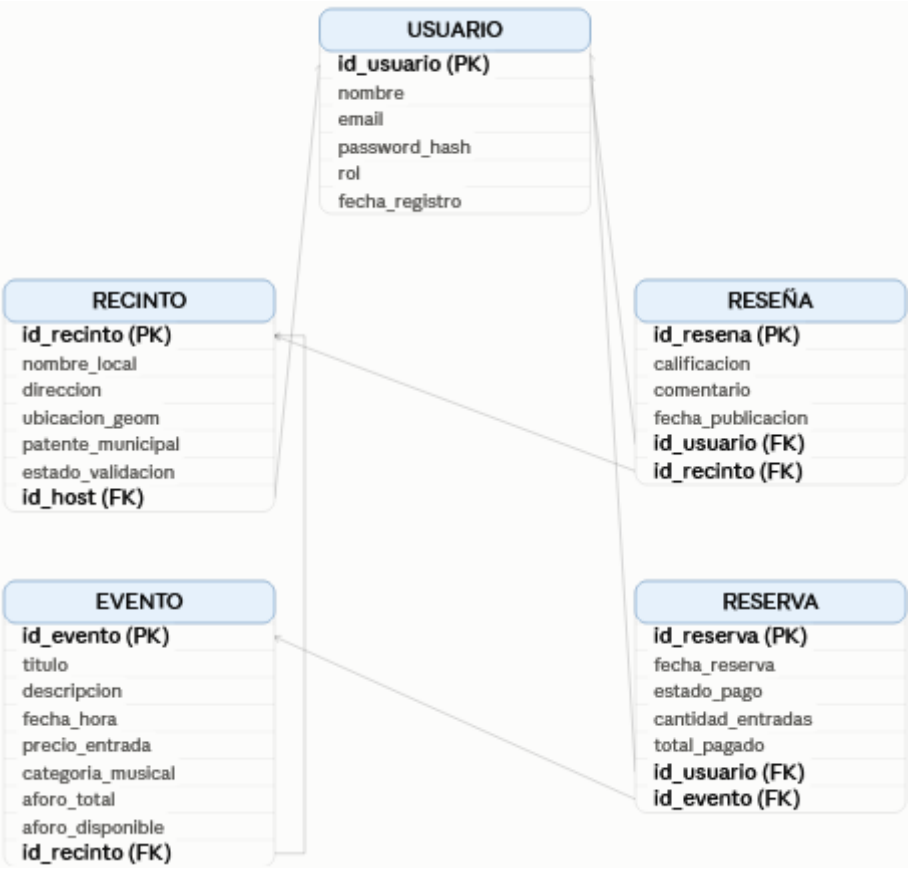


Figura 6: Esquema Relacional Lógico

3.3.1. Tabla: USUARIO

Cuadro 1: Diccionario de datos para la tabla USUARIO.

Nombre Atributo	PK/FK	Tipo de Dato	NULL/NOT NULL	Valor por Defecto	Restricciones / Descripción
id_usuario	PK	BIGSERIAL	NOT NULL	Auto	Identificador único del usuario
nombre	-	VARCHAR(100)	NOT NULL	-	Nombre completo
email	-	VARCHAR(150)	NOT NULL	-	UNIQUE, Email de acceso
password_hash	-	VARCHAR(255)	NOT NULL	-	Hash encriptado bcrypt
rol	-	VARCHAR(20)	NOT NULL	-	CHECK (rol IN ('CLIENTE', 'HOST', 'ADMIN'))
fecha_registro	-	TIMESTAMP	-	CURRENT_TIMESTAMP	Fecha de creación de la cuenta

3.3.2. Tabla: RECINTO

Cuadro 2: Diccionario de datos para la tabla RECINTO.

Nombre Atributo	PK/FK	Tipo de Dato	NULL/NOT NULL	Valor por Defecto	Restricciones / Descripción
id_recinto	PK	BIGSERIAL	NOT NULL	Auto	Identificador único del recinto
nombre_local	-	VARCHAR(120)	NOT NULL	-	Nombre comercial del local
direccion	-	VARCHAR(200)	NOT NULL	-	Dirección física textual
ubicacion_geom	-	GEOMETRY(POINT, 4326)	NOT NULL	-	Coordenadas GPS (PostGIS, WGS84)
patente_municipal	-	VARCHAR(50)	NOT NULL	-	Documento de habilitación legal
estado_validacion	-	VARCHAR(20)	-	'PENDIENTE'	CHECK (estado IN ('PENDIENTE','APROBADO','RECHAZADO'))
id_host	FK	BIGINT	NOT NULL	-	Referencia a USUARIO(id_usuario)

3.3.3. Tabla: EVENTO

Cuadro 3: Diccionario de datos para la tabla EVENTO.

Nombre Atributo	PK/FK	Tipo de Dato	NULL/NOT NULL	Valor por Defecto	Restricciones / Descripción
id_evento	PK	BIGSERIAL	NOT NULL	Auto	Identificador único del evento
titulo	-	VARCHAR(150)	NOT NULL	-	Nombre del evento
descripcion	-	TEXT	-	-	Detalle del evento
fecha_hora	-	TIMESTAMP	NOT NULL	-	Fecha y hora programada
precio_entrada	-	NUMERIC(10,2)	-	0.00	CHECK (precio_entrada >= 0)
categoria_musical	-	VARCHAR(50)	NOT NULL	-	Estilo de música del evento
aforo_total	-	INT	NOT NULL	-	CHECK (aforo_total > 0)
aforo_disponible	-	INT	NOT NULL	-	CHECK (aforo_disponible >= 0 AND aforo_disponible <= aforo_total)
id_recinto	FK	BIGINT	NOT NULL	-	Referencia a RECINTO(id_recinto)

3.3.4. Tabla: RESERVA

Cuadro 4: Diccionario de datos para la tabla RESERVA.

Nombre Atributo	PK/FK	Tipo de Dato	NULL/NOT NULL	Valor por Defecto	Restricciones / Descripción
id_reserva	PK	BIGSERIAL	NOT NULL	Auto	Código identificador del ticket
fecha_reserva	-	TIMESTAMP	-	CURRENT_TIMESTAMP	Momento de la transacción
estado_pago	-	VARCHAR(20)	NOT NULL	-	CHECK (estado_pago IN ('PENDIENTE','PAGADO','CANCELADO'))
cantidad_entradas	-	INT	-	1	CHECK (cantidad_entradas > 0 AND cantidad_entradas <= 5)
total_pagado	-	NUMERIC(10,2)	NOT NULL	-	Monto acumulado procesado
id_usuario	FK	BIGINT	NOT NULL	-	Referencia a USUARIO(id_usuario)
id_evento	FK	BIGINT	NOT NULL	-	Referencia a EVENTO(id_evento)

3.3.5. Tabla: RESEÑA

Cuadro 5: Diccionario de datos para la tabla RESEÑA.

Nombre Atributo	PK/FK	Tipo de Dato	NULL/NOT NULL	Valor por Defecto	Restricciones / Descripción
id_resena	PK	BIGSERIAL	NOT NULL	Auto	Identificador único de la reseña
calificacion	-	INT	NOT NULL	-	CHECK (calificacion >= 1 AND calificacion <= 5)
comentario	-	TEXT	-	-	Opinión textual del usuario sobre el local
fecha_publicacion	-	TIMESTAMP	-	CURRENT_TIMESTAMP	Fecha y hora en que se emite la reseña
id_usuario	FK	BIGINT	NOT NULL	-	Referencia a USUARIO(id_usuario)
id_recinto	FK	BIGINT	NOT NULL	-	Referencia a RECINTO(id_recinto)

3.4. Restricciones de Integridad Referencial y Supuestos

Restricciones de Integridad Referencial:

- **RECINTO → USUARIO (Host):** `ON DELETE RESTRICT / ON UPDATE CASCADE`. No se permite la eliminación de una cuenta de Host Comercial si mantiene recintos registrados asociados.
- **EVENTO → RECINTO:** `ON DELETE CASCADE / ON UPDATE CASCADE`. Si un recinto es eliminado del sistema, se cancelan y eliminan en cascada todos sus eventos asociados.
- **RESERVA → EVENTO:** `ON DELETE RESTRICT / ON UPDATE CASCADE`. Se impide la eliminación física de un evento si ya existen reservas o tickets emitidos a usuarios, asegurando el cumplimiento legal y financiero.
- **RESERVA → USUARIO:** `ON DELETE RESTRICT / ON UPDATE CASCADE`. No se permite dar de baja a un cliente mientras tenga cualquier reserva relacionada.
- **RESEÑA → RECINTO / USUARIO:** `ON DELETE CASCADE / ON UPDATE CASCADE`. Si un usuario o un recinto es eliminado del sistema, se eliminan automáticamente en cascada sus reseñas asociadas para mantener la consistencia.

Supuestos del Modelo de Negocio:

- **Límite transaccional:** Para prevenir el acaparamiento y la reventa por *bots*, un usuario asistente solo puede adquirir un máximo de 5 entradas por transacción en una misma reserva.
- **Soporte Geoespacial:** Las coordenadas geográficas de los recintos se almacenan usando el tipo de dato espacial `GEOMETRY(POINT, 4326)` del estándar WGS84, permitiendo realizar consultas de proximidad indexadas mediante PostGIS.
- **Aprobación Legal Obligatoria:** Un recinto registrado por un Host Comercial permanece en estado `'PENDIENTE'` sin posibilidad de publicar eventos hasta que un Administrador valide la patente municipal y cambie su estado a `'APROBADO'`.
- **Control de Concurrencia:** El descuento del atributo `aforo_disponible` en la tabla `EVENTO` se realiza dentro de una transacción atómica bloqueando el registro mediante sintaxis SQL (`SELECT ... FOR UPDATE`) para garantizar aislamiento (ACID) y evitar la sobreventa ante compras simultáneas.
- **Validación de Reseñas:** La regla de negocio que dicta que un usuario solo pueda reseñar un local si efectivamente asistió al evento, se validará de manera estricta a nivel de capa de servicio (Backend), verificando la existencia previa de una reserva `'PAGADO'` pasada antes de ejecutar el `INSERT` en la tabla `RESEÑA`.

4. Consultas SQL y Álgebra Relacional

A continuación, se presenta el catálogo de las 10 consultas obligatorias diseñadas y documentadas para el sistema. Estas operan de forma fidedigna sobre el esquema DDL exacto del proyecto y demuestran el uso de operadores avanzados requeridos por la rúbrica. Cabe destacar que estas diez consultas están documentadas a nivel de diseño conceptual en este reporte, no estando necesariamente integradas en su totalidad en el backend ni en los scripts `06_consultas.sql` del repositorio actual en esta etapa.

4.1. Selección Básica (Filtro Múltiple)

Descripción: Eventos con precio de entrada mayor a $10.000 y que aún tengan aforo disponible.

Álgebra Relacional:

$$\pi_{titulo, fecha_hora, precio_entrada}(\sigma_{precio_entrada > 10000 \wedge aforo_disponible > 0}(\text{Evento}))$$

```
SELECT titulo, fecha_hora, precio_entrada
FROM evento
WHERE precio_entrada > 10000 AND aforo_disponible > 0;
```

4.2. JOIN de 2 Tablas

Descripción: Locales y eventos asociados, pero solo para aquellos recintos cuyo estado de validación esté aprobado.

Álgebra Relacional:

$$\pi_{nombre_local, titulo}(\sigma_{estado_validacion = 'APROBADO'}(\text{Recinto}) \bowtie \text{Evento})$$

```
SELECT r.nombre_local, e.titulo
FROM recinto r
INNER JOIN evento e ON r.id_recinto = e.id_recinto
WHERE r.estado_validacion = 'APROBADO';
```

4.3. JOIN de 3 Tablas (Reservas Pagadas)

Descripción: Listado de usuarios, el evento al que asistirán y la cantidad de entradas, filtrando solo las reservas con pago completado.

Álgebra Relacional:

$$\pi_{nombre, titulo, cantidad_entradas}\Big(\text{Usuario} \bowtie \sigma_{estado_pago = 'PAGADO'}(\text{Reserva}) \bowtie \text{Evento} \Big)$$

```
SELECT u.nombre, e.titulo, r.cantidad_entradas
FROM usuario u
INNER JOIN reserva r ON u.id_usuario = r.id_usuario
INNER JOIN evento e ON r.id_evento = e.id_evento
WHERE r.estado_pago = 'PAGADO';
```

4.4. Agrupación y Funciones de Agregación (GROUP BY)

Descripción: Resumen por recinto indicando cuántos eventos tiene, la capacidad total de los mismos y el precio promedio de las entradas.

Álgebra Relacional:

$$\gamma_{id_recinto,nombre_local,\text{COUNT}(id_evento)\to total,\text{SUM}(aforo_total)\to cap,\text{AVG}(precio_entrada)\to prom}\Big(\text{Recinto} \bowtie_{\text{LEFT}} \text{Evento}\Big)$$

```
SELECT r.id_recinto, r.nombre_local, COUNT(e.id_evento) AS
   total_eventos,
       COALESCE(SUM(e.aforo_total), 0) AS capacidad_total_eventos
          ,
       ROUND(COALESCE(AVG(e.precio_entrada), 0), 2) AS
          precio_promedio
FROM recinto r
LEFT JOIN evento e ON r.id_recinto = e.id_recinto
GROUP BY r.id_recinto, r.nombre_local
ORDER BY total_eventos DESC;
```

4.5. Ranking con Filtro Post-Agregación (HAVING)

Descripción: Locales que poseen al menos 1 reseña, ordenados por su calificación promedio para destacar los favoritos del público.

Álgebra Relacional:

$$\pi_{nombre_local,resenas,prom}\Big(\sigma_{resenas\geq 1}\Big(\gamma_{id_recinto,nombre_local,\text{COUNT}(id_resena)\to resenas,\text{AVG}(calificacion)\to prom}\Big(\text{Recinto} \bowtie \text{Reseña}\Big)\Big)\Big)$$

```
SELECT r.nombre_local, COUNT(res.id_resena) AS cantidad_resenas,
       ROUND(AVG(res.calificacion), 1) AS promedio_real
FROM recinto r
INNER JOIN resena res ON r.id_recinto = res.id_recinto
GROUP BY r.id_recinto, r.nombre_local
HAVING COUNT(res.id_resena) >= 1
ORDER BY promedio_real DESC, cantidad_resenas DESC
LIMIT 5;
```

4.6. Búsqueda Geoespacial (PostGIS)

Descripción: Calcula la distancia real en metros desde un punto fijo y filtra recintos en un radio caminable de 1.5 kilómetros.

Álgebra Relacional:

$$\pi_{nombre_local,direccion,ST_X(ubicacion_geom)\rightarrow lng,ST_Y(ubicacion_geom)\rightarrow lat,\text{ROUND}(ST_Distance(...)::numeric)\rightarrow distancia_metros}$$
$$\sigma_{ST_DWithin(ubicacion_geom::geography,Punto,1500)}(\text{Recinto})$$
$$\Big)$$

Nota: Punto = ST_SetSRID(ST_MakePoint(−71,624, −33,044), 4326) :: geography y el cálculo completo proyectado es ST_Distance(ubicacion_geom :: geography, Punto) :: numeric.

```
SELECT r.nombre_local, r.direccion, ST_X(r.ubicacion_geom) AS lng
    ,
        ST_Y(r.ubicacion_geom) AS lat,
        ROUND(ST_Distance(r.ubicacion_geom::geography,
        ST_SetSRID(ST_MakePoint(-71.624, -33.044), 4326)::
           geography)::numeric) AS distancia_metros
FROM recinto r
WHERE ST_DWithin(r.ubicacion_geom::geography,
      ST_SetSRID(ST_MakePoint(-71.624, -33.044), 4326)::geography
         , 1500)
ORDER BY distancia_metros ASC;
```

4.7. Teoría de Conjuntos (INTERSECT)

Descripción: Intersección para encontrar usuarios muy involucrados: tienen reservas pagadas Y además han dejado alguna reseña en la plataforma.

Álgebra Relacional:

$$\pi_{id_usuario,nombre,email}(\text{Usuario} \bowtie \sigma_{estado_pago='PAGADO'}(\text{Reserva}))$$
$$\cap$$
$$\pi_{id_usuario,nombre,email}(\text{Usuario} \bowtie \text{Reseña})$$

```
SELECT u.id_usuario, u.nombre, u.email
FROM usuario u
INNER JOIN reserva r ON u.id_usuario = r.id_usuario
WHERE r.estado_pago = 'PAGADO'
INTERSECT
SELECT u.id_usuario, u.nombre, u.email
FROM usuario u
INNER JOIN resena res ON u.id_usuario = res.id_usuario;
```

4.8. Operación Matemática en Predicado (WHERE)

Descripción: Alerta de Çasi Agotado"(FOMO) detectando eventos donde la razón de aforo disponible respecto al aforo total es igual o inferior al 20 %.

Álgebra Relacional:

$$\pi_{id_evento,titulo,nombre_local,disp,total,pct}\Big(\sigma_{disp>0\wedge(disp/total)\leq 0{,}20}(\text{Evento} \bowtie \text{Recinto})\Big)$$

Nota: disp representa aforo_disponible, total representa aforo_total y pct representa porcentaje_disponible.

```
SELECT e.id_evento, e.titulo, r.nombre_local, e.aforo_disponible,
    e.aforo_total,
       ROUND((e.aforo_disponible::numeric / e.aforo_total::
          numeric) * 100, 1) AS porcentaje_disponible
FROM evento e
INNER JOIN recinto r ON e.id_recinto = r.id_recinto
WHERE e.aforo_disponible > 0
  AND (e.aforo_disponible::numeric / e.aforo_total::numeric) <=
     0.20
ORDER BY porcentaje_disponible ASC;
```

4.9. Filtro Temporal Dinámico (CURRENT_TIMESTAMP)

Descripción: Cartelera de próximos eventos. Omite fiestas pasadas comparando la fecha de la base de datos contra el reloj del sistema.

Álgebra Relacional:

$$\pi_{titulo,nombre_local,fecha_hora}(\sigma_{fecha_hora\geq CURRENT_TIMESTAMP}(\text{Evento} \bowtie \text{Recinto}))$$

```
SELECT e.titulo, r.nombre_local, e.fecha_hora
FROM evento e
INNER JOIN recinto r ON e.id_recinto = r.id_recinto
WHERE e.fecha_hora >= CURRENT_TIMESTAMP
ORDER BY e.fecha_hora ASC;
```

4.10. Teoría de Conjuntos (EXCEPT)

Descripción: Lista de usuarios inactivos que se registraron en la aplicación pero a los cuales no se les registra ninguna reserva pagada exitosamente.

Álgebra Relacional:

$$\pi_{id_usuario,nombre,email}(\text{Usuario}) - \pi_{id_usuario,nombre,email}(\text{Usuario} \bowtie \sigma_{estado_pago='PAGADO'}(\text{Reserva}))$$

```
SELECT id_usuario, nombre, email
FROM usuario
EXCEPT
SELECT u.id_usuario, u.nombre, u.email
FROM usuario u
INNER JOIN reserva res ON u.id_usuario = res.id_usuario
WHERE res.estado_pago = 'PAGADO';
```

5. Propuesta de migración de PostgreSQL a MongoDB

5.1. Alcance y estado de la migración

La base de datos implementada y descrita en las secciones anteriores es relacional, utiliza PostgreSQL y contempla soporte geoespacial mediante PostGIS. Esta sección presenta una propuesta académica para trasladar ese modelo a MongoDB, en el contexto de la Entrega 2. La inclusión de este análisis no significa que el backend o la base de datos del proyecto ya hayan sido migrados.

Los objetivos son rediseñar las relaciones como documentos y colecciones según los patrones de consulta de DondeE, definir la conversión de los tipos SQL a BSON y proponer controles para validar los datos transferidos.

5.2. Decisiones de modelado documental

La migración no consiste en reemplazar cada tabla por una colección con la misma estructura. Las decisiones siguientes consideran el crecimiento esperado de los datos, las consultas de la aplicación y el historial que debe conservarse.

Origen	**Destino propuesto**	**Justificación**
USUARIO	Colección `usuarios`.	Se conserva independiente porque las reservas y reseñas de un usuario pueden crecer. No se incrustan credenciales en otros documentos.
RECINTO	Colección `recintos`, con ubicación GeoJSON y una referencia extendida acotada al host.	Permite búsquedas geoespaciales y mostrar información básica del host sin duplicar datos privados.
EVENTO	Colección `eventos`, con una referencia a `id_recinto` y un snapshot limitado del recinto.	Permite filtrar la cartelera independientemente del recinto y mostrar su ubicación con menos consultas. Los cambios de dirección o ubicación requieren actualizar el snapshot.
RESERVA	Colección `reservas`, independiente, con un snapshot de la compra.	El historial puede crecer sin límite. El snapshot conserva los datos de la compra aunque cambie el evento o el recinto.
RESEÑA	Colección `resenas`, independiente. En el recinto se pueden mantener agregados derivados.	Permite paginar reseñas sin cargar un arreglo potencialmente ilimitado. `rating_promedio` y `resenas_count` son derivados, no columnas del DDL relacional.

Los documentos conservan los identificadores de origen (`id_usuario`, `id_recinto`, etc.) para facilitar la trazabilidad y la compatibilidad con los contratos actuales de la API.

MongoDB añade además el campo `_id`. Las referencias numéricas no crean claves foráneas automáticamente; la aplicación y el proceso de migración deben validar su integridad.

5.3. Conversión de tipos SQL a BSON

- **BIGSERIAL y BIGINT:** se conservan como BSON `Long` en los campos `id_*`. En Extended JSON v2 se representan como {`"$numberLong":"1"`}.
- **Identificador MongoDB:** cada documento cuenta con un `_id` de tipo `ObjectId`, separado del identificador relacional conservado.
- **GEOMETRY(POINT, 4326):** se transforma a GeoJSON `Point`. El arreglo de coordenadas usa el orden longitud, latitud: `[longitud, latitud]`.
- **TIMESTAMP:** se convierte a BSON `Date`. Como el esquema SQL no especifica zona horaria, el ETL debe verificar cómo se interpretan las fechas de origen antes de convertirlas a UTC.
- **NUMERIC(10,2):** se convierte a `Decimal128`, evitando usar valores binarios de punto flotante para montos monetarios.
- **Campos nulos o ausentes:** se conserva una regla consistente para valores opcionales como `descripcion` y `comentario`; no se inventan datos.

Los siguientes ejemplos usan MongoDB Extended JSON v2. Los identificadores y algunos valores son ilustrativos: no reemplazan una exportación real de todas las filas de PostgreSQL.

5.4. Ejemplos de documentos JSON / BSON

5.4.1. Colección `recintos`

La ubicación se representa como GeoJSON. El promedio de calificación y la cantidad de reseñas, si se almacenan, se calculan a partir de la colección `resenas`.

```
{
  "_id": { "$oid": "67069b12a01b4c0001000001" },
  "id_recinto": { "$numberLong": "1" },
  "nombre_local": "Club El Huevo",
  "direccion": "Blanco 1386, Valparaiso",
  "patente_municipal": "PAT-VALP-001",
  "estado_validacion": "APROBADO",
  "ubicacion": {
    "type": "Point",
    "coordinates": [-71.6245, -33.0425]
  },
  "host": {
    "id_usuario": { "$numberLong": "9" },
    "nombre": "Host Valparaiso Puerto SpA"
  },
  "rating_promedio": 5.0,
  "resenas_count": 1
}
```

5.4.2. Colección `eventos`

El evento mantiene la referencia al recinto y un snapshot acotado para facilitar su visualización en el mapa y en la cartelera.

```
{
  "_id": { "$oid": "67069b12a01b4c0002000001" },
  "id_evento": { "$numberLong": "1" },
  "titulo": "Noche Estelar en Club El Huevo",
  "descripcion": "Evento de fin de semana",
  "fecha_hora": { "$date": "2026-10-11T01:30:00.000Z" },
  "precio_entrada": { "$numberDecimal": "8000.00" },
  "categoria_musical": "Hits & Bailable",
  "aforo_total": 250,
  "aforo_disponible": 35,
  "id_recinto": { "$numberLong": "1" },
  "recinto": {
    "nombre_local": "Club El Huevo",
    "direccion": "Blanco 1386, Valparaiso",
    "ubicacion": {
      "type": "Point",
      "coordinates": [-71.6245, -33.0425]
    }
  },
  "fecha_hora_origen_zona": "2026-10-10T22:30:00",
  "zona_horaria_origen": "America/Santiago"
}
```

La fecha UTC del ejemplo es ilustrativa. La conversión real debe basarse en la fecha y zona horaria verificadas en la extracción, no en una suposición del ETL.

5.4.3. Colección `usuarios`

Las credenciales permanecen en esta colección y no deben exponerse en consultas públicas ni incluirse en capturas o logs.

```
{
  "_id": { "$oid": "67069b12a01b4c0003000001" },
  "id_usuario": { "$numberLong": "1" },
  "nombre": "Usuario de ejemplo",
  "email": "usuario.demo@example.com",
  "password_hash": "<hash_bcrypt_de_origen>",
  "rol": "CLIENTE",
  "fecha_registro": { "$date": "2026-09-20T17:30:00.000Z" },
  "zona_horaria_origen": "America/Santiago"
}
```

El texto entre ángulos es un marcador y no un hash utilizable. En una migración real se conserva el hash de origen sin imprimirlo en documentación o registros.

5.4.4. Colección `reservas`

La reserva conserva los valores de la transacción y una copia de los datos históricos de la compra. No se debe recalcular el monto pagado usando el precio vigente del evento.

```
{
  "_id": { "$oid": "67069b12a01b4c0004000001" },
  "id_reserva": { "$numberLong": "1" },
  "fecha_reserva": { "$date": "2026-10-19T22:55:48.616Z" },
  "estado_pago": "PAGADO",
  "cantidad_entradas": 2,
  "total_pagado": { "$numberDecimal": "20000.00" },
  "id_usuario": { "$numberLong": "1" },
  "id_evento": { "$numberLong": "1" },
  "compra_snapshot": {
    "titulo_evento": "Noche Estelar en Club El Huevo",
    "fecha_hora_evento": { "$date": "2026-10-11T01:30:00.000Z" },
    "nombre_local": "Club El Huevo"
  },
  "zona_horaria_origen": "America/Santiago"
}
```

El documento relacional guarda `total_pagado`, pero no registra necesariamente el precio unitario histórico. El proceso debe conservar el monto extraído y reportar diferencias que requieran revisión, sin alterar el dato de origen.

5.4.5. Colección `resenas`

Las reseñas se mantienen separadas para permitir consultas paginadas y evitar arreglos que crezcan sin límite dentro de un recinto.

```
{
  "_id": { "$oid": "67069b12a01b4c0005000001" },
  "id_resena": { "$numberLong": "1" },
  "id_recinto": { "$numberLong": "1" },
  "id_usuario": { "$numberLong": "2" },
  "calificacion": 5,
  "comentario": "Buena experiencia en el local.",
  "fecha_publicacion": { "$date": "2026-10-17T00:00:00.000Z" },
  "zona_horaria_origen": "America/Santiago",
  "autor_snapshot": {
    "nombre": "Usuario de ejemplo"
  }
}
```

5.5. Índices y consultas geoespaciales

MongoDB requiere un índice `2dsphere` para búsquedas sobre coordenadas GeoJSON. Los siguientes índices son una propuesta inicial según las consultas descritas en este informe:

```
db.recintos.createIndex({ ubicacion: "2dsphere" });
db.eventos.createIndex({ "recinto.ubicacion": "2dsphere" });

db.usuarios.createIndex({ email: 1 }, { unique: true });
db.recintos.createIndex({ id_recinto: 1 }, { unique: true });
db.eventos.createIndex({ id_evento: 1 }, { unique: true });
```

```
db.reservas.createIndex({ id_reserva: 1 }, { unique: true });
db.resenas.createIndex({ id_resena: 1 }, { unique: true });

db.eventos.createIndex({ fecha_hora: 1, categoria_musical: 1 });
db.reservas.createIndex({ id_usuario: 1, fecha_reserva: -1 });
db.resenas.createIndex({ id_recinto: 1, fecha_publicacion: -1 });
```

Ejemplo de búsqueda de recintos en un radio de 3 kilómetros desde un punto de referencia. El índice `2dsphere` opera en metros para esta consulta:

```
db.recintos.find({
  ubicacion: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [-71.6248, -33.0445]
      },
      $maxDistance: 3000
    }
  }
}, {
  _id: 0,
  id_recinto: 1,
  nombre_local: 1,
  direccion: 1
});
```

Los índices ocupan almacenamiento y hacen más costosas las escrituras. Su utilidad se debe validar con planes de ejecución y datos representativos; no se afirma que MongoDB sea automáticamente más rápido que PostgreSQL.

5.6. Proceso propuesto de migración y validación

Se propone un proceso ETL (extracción, transformación y carga) en las siguientes etapas:

1. Inventariar las cinco tablas y sus columnas en el esquema de PostgreSQL.
2. Extraer todas las filas y conservar sus claves primarias y referencias foráneas como campos `id_*`.
3. Convertir geometrías a GeoJSON con coordenadas en orden longitud-latitud; validar el SRID y los valores.
4. Convertir fechas a BSON `Date` después de confirmar la zona horaria de origen.
5. Convertir montos `NUMERIC` a `Decimal128`; comprobar restricciones de aforo, cantidad de entradas y calificaciones.
6. Generar los agregados derivados de las reseñas y documentar su fuente y forma de actualización.
7. Cargar primero usuarios y recintos; luego eventos, reservas y reseñas, validando que las referencias apunten a documentos existentes.

8. Comparar los conteos de origen y destino, revisar errores y filas rechazadas, y ejecutar la carga de forma repetible sin duplicar documentos.

Como referencia, el script de datos de demostración espera 12 usuarios, 42 recintos, 126 eventos, 4 reservas y 5 reseñas cuando se ejecuta una sola vez sobre una base vacía. Estos son conteos del conjunto de demostración, no límites del sistema; en una migración deben compararse los valores efectivamente extraídos.

MongoDB no crea claves foráneas automáticamente. La aplicación debe validar referencias y reglas de negocio. Para una compra, la actualización del aforo debe comprobar que queden entradas suficientes y ejecutarse de forma atómica con la inserción de la reserva. Las operaciones que modifican varios documentos pueden usar una transacción en un replica set; Atlas proporciona ese tipo de entorno, pero la configuración local también debe considerarlo.

5.7. Seguridad y estado de implementación

La cadena de conexión de MongoDB debe configurarse como variable de entorno, por ejemplo `MONGODB_URI`, y no escribirse en el informe, el repositorio ni los logs. Las credenciales reales deben permanecer fuera del control de versiones y contar con permisos mínimos.

Esta sección documenta un diseño y un procedimiento propuesto. Hasta que se implemente y pruebe el ETL, los índices y el adaptador del backend, la base de datos del proyecto sigue siendo la descrita en las secciones de PostgreSQL/PostGIS.

A. Guía de usabilidad y navegación

A.1. Propósito y alcance

Esta guía describe los criterios de usabilidad propuestos para que una persona pueda descubrir oferta nocturna formal, revisar sus antecedentes y avanzar hacia una reserva con información clara. El diseño prioriza el uso desde dispositivos móviles, coherente con el requerimiento RNF04. Las pantallas y estados de esta sección constituyen evidencia de diseño y prototipado; no implican que todas las funciones descritas estén implementadas o conectadas a servicios reales en la versión actual.

A.2. Usuarios y objetivos

- **Usuario asistente:** encontrar locales y eventos cercanos, comparar información relevante y comprender cómo continuar hacia una reserva.
- **Host Comercial:** consultar y gestionar la información de su recinto y sus eventos, sujeto a la validación administrativa definida por el sistema.
- **Administrador:** revisar antecedentes de los recintos y comunicar con claridad el estado de su validación.

A.3. Flujo principal del usuario asistente

1. **Explorar:** abrir el mapa y elegir una zona mediante la ubicación disponible o una búsqueda manual.
2. **Filtrar:** indicar el radio de búsqueda y, opcionalmente, ajustar criterios como fecha, categoría, precio o calificación.
3. **Comparar:** seleccionar un marcador o resultado para revisar los datos del local y del evento, como dirección, horario, precio y disponibilidad.
4. **Continuar:** desde el detalle, iniciar el flujo de reserva propuesto y revisar la cantidad de entradas y el total antes de confirmar.
5. **Confirmar:** mostrar el resultado de la operación y, si corresponde a la versión implementada, presentar el comprobante o ticket asociado.

El flujo de reserva, pago y billetera se presenta como comportamiento esperado del diseño. Debe rotularse como simulado o propuesto cuando no esté respaldado por una integración real.

A.4. Criterios para las pantallas

- **Mapa y búsqueda:** mantener el buscador visible, permitir cambiar de zona sin depender exclusivamente del GPS y diferenciar visualmente locales o eventos disponibles.
- **Resultados:** resumir la información necesaria para comparar opciones y ofrecer una acción evidente para abrir el detalle.

- **Detalle de local o evento:** priorizar nombre, ubicación, fecha y hora, precio y disponibilidad; dejar reseñas y datos complementarios en una jerarquía secundaria.
- **Reserva:** presentar cantidad, precio unitario y total antes de confirmar; indicar de forma visible si el proceso de pago es una simulación.
- **Confirmación o entradas:** comunicar el estado de la operación. Mostrar un ticket o código únicamente si existe en el prototipo; en caso contrario, rotular esa pantalla como propuesta.

A.5. Mensajes y estados

La interfaz debe entregar retroalimentación breve y orientar la acción siguiente. Se proponen los siguientes estados:

- **Sin resultados:** "No se encontraron eventos en esta zona. Prueba ampliar el radio o cambiar los filtros."
- **Ubicación no disponible:** "No pudimos acceder a tu ubicación. Puedes buscar una zona manualmente."
- **Sin disponibilidad:** "Este evento no tiene entradas disponibles."
- **Carga o verificación:** "Estamos verificando la disponibilidad. Esto puede tardar unos segundos."
- **Error de conexión:** "No pudimos cargar la información. Revisa tu conexión e inténtalo nuevamente."
- **Operación completada:** "Solicitud procesada." El mensaje debe especificar si la reserva quedó confirmada o si solo se simuló el proceso.

A.6. Navegación, accesibilidad y adaptación

- Mantener una navegación consistente entre mapa, resultados, detalle y reserva, con una acción clara para volver al paso anterior.
- Usar etiquetas legibles, contraste suficiente y botones con áreas táctiles cómodas; no comunicar disponibilidad o categorías solo mediante color.
- En pantallas pequeñas, priorizar la búsqueda y el mapa, y presentar los detalles en paneles que no oculten las acciones principales.
- Permitir continuar sin permisos de ubicación, ofreciendo búsqueda manual como alternativa.
- En escritorio, conservar la misma jerarquía y permitir que el mapa y la información de resultados se vean simultáneamente cuando el espacio lo permita.

A.7. Criterios de revisión del prototipo

Para evaluar el diseño, una persona debe poder identificar cómo buscar, abrir un resultado y localizar la acción para continuar sin instrucciones externas. También debe poder reconocer si una operación está cargando, terminó, falló o representa solo una simulación. Estos criterios son objetivos de evaluación del prototipo; no se presentan como resultados de pruebas de usuario realizadas.

A.8. Mockups y capturas de pantalla

Reemplazar cada recuadro por la imagen correspondiente. Se recomienda usar capturas o mockups legibles, rotular cada figura y mantener visibles los controles principales. Las pantallas de reserva, confirmación y billetera deben identificarse como propuestas o simulaciones si aún no están implementadas.

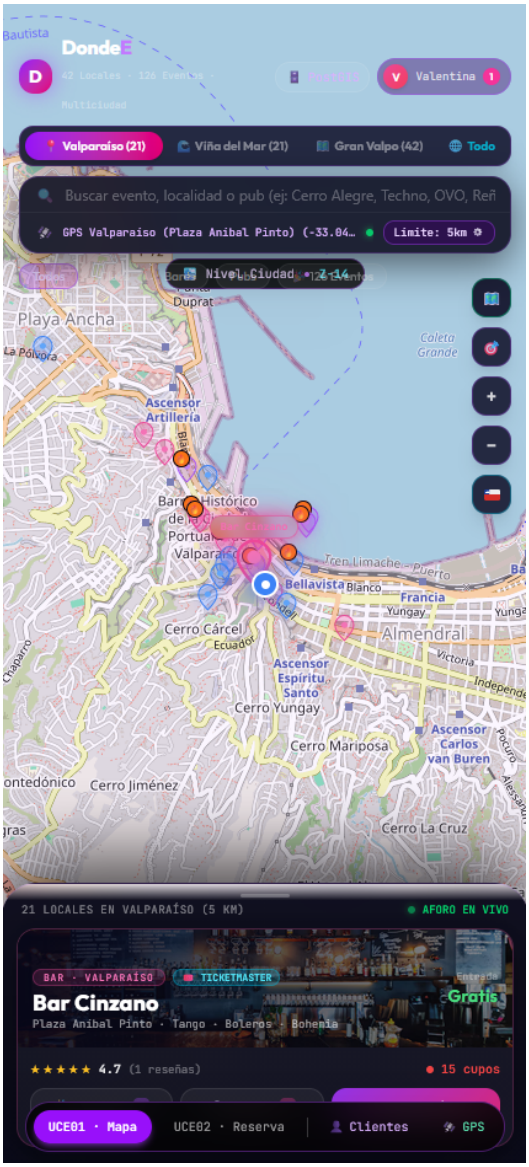


Figura 7: Propuesta de pantalla de exploración de eventos en el mapa.

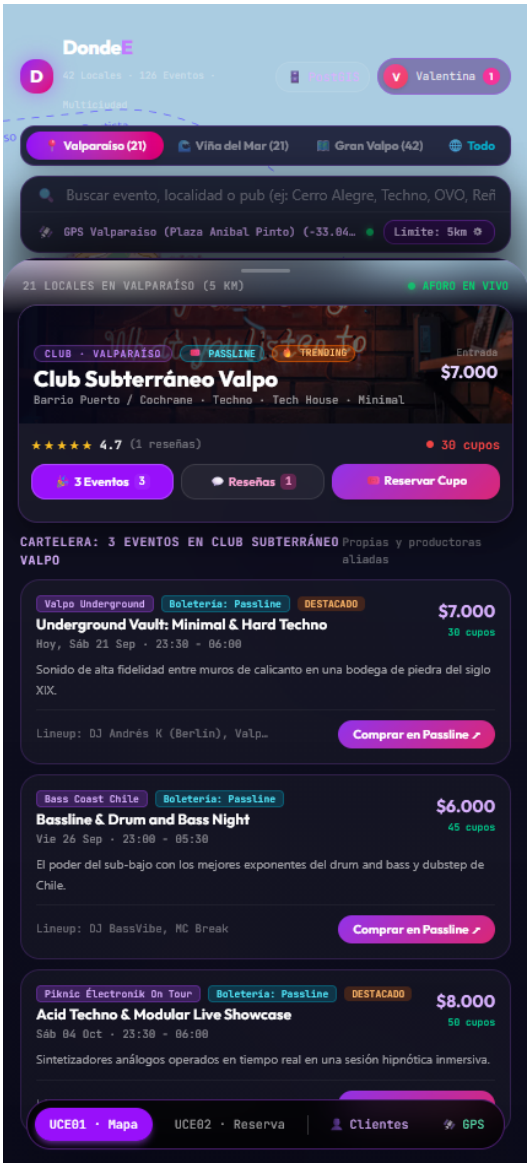


Figura 8: Propuesta de resultados y detalle de un evento o recinto.

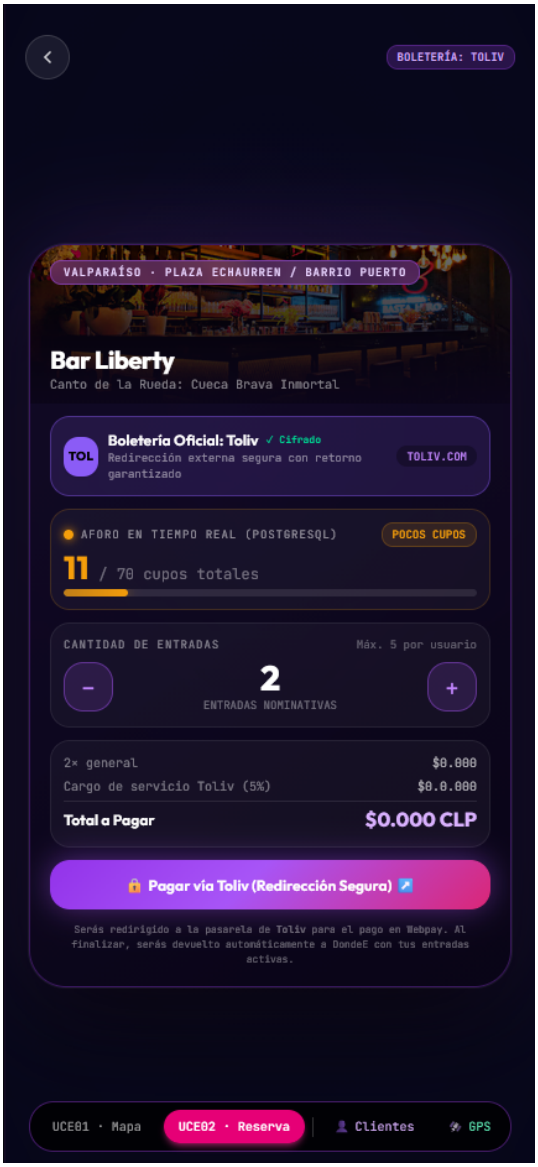


Figura 9: Propuesta del flujo de reserva. Ejemplo simulado.

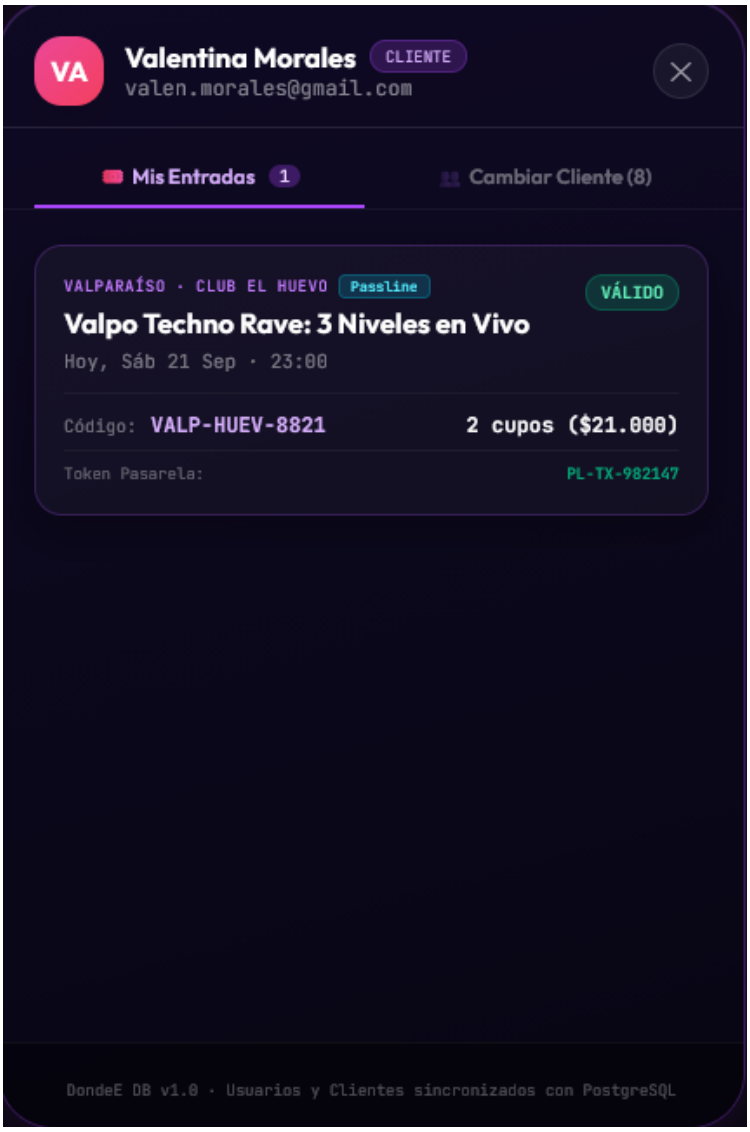


Figura 10: Propuesta de confirmación o billetera.